AWS EC2 Deployment

Free-Tier Web Server Lab Guide

Step-by-Step Instructions from Scratch

What You Will Learn

- Create and secure a Free-Tier AWS account
- Launch an EC2 instance inside a custom VPC
- Configure security groups for SSH and RDP
- Connect remotely via SSH (Ubuntu) and RDP (Windows)
- Install Apache/Nginx, deploy a page, and verify it publicly

Estimated Time: 60 – 90 minutes

Skill Level: Beginner to Intermediate

Task 1 — Create a Free-Tier AWS Account

Step 1.1 Prerequisites

Before you begin, make sure you have the following:

- A valid email address not already associated with an AWS account.
- A credit or debit card for identity verification (not charged if you stay within Free Tier limits).
- A mobile phone number that can receive an SMS or voice call.
- Basic familiarity with using a web browser.

⚠ **Never share your AWS root account credentials with anyone. You will create a separate IAM user for daily use in Step 1.3.**

Step 1.2 Sign Up for AWS

1. Open a browser and go to: https://aws.amazon.com/free
2. Click Create a Free Account (top-right corner).
3. Enter your email address and an AWS account name, then click Verify email address.
4. Enter the one-time code sent to your email and click Verify.
5. Create a strong root user password and confirm it.
6. Choose account type: Personal (individual use) or Business.
7. Enter your full name, phone number, country, and address; accept the AWS Customer Agreement.
8. Enter payment card details. AWS places a small temporary authorization hold, not a charge.
9. Complete phone verification: choose SMS or voice call, solve the CAPTCHA, and enter the code you receive.
10. Select the Basic Support – Free plan.
11. Click Complete Sign Up. A confirmation email arrives once the account is activated (usually within a few minutes).

☑ **Signed up!** You'll receive a welcome email from AWS once your account is fully active.

Step 1.3 First Sign-In and Basic Hardening

12. Go to https://aws.amazon.com/console and sign in as the root user with your email and password.
13. In the console search bar, type IAM and open the IAM service.
14. Click Add MFA for the root user and register an authenticator app or security key (**optional**).
15. Go to Users → Create user. Name it **admin-user** (**Optional, can provide name as per your convenience**), attach the AdministratorAccess policy, and set sign-in credentials.
16. Sign out of the root account and sign back in as admin-user (**Optional, can provide name as per your convenience**) for the rest of this guide.

17. Provide the following permissions: There are 8 predefined, and one is custom-based. The custom-based will be created at **Step 2.2.5 Connect via RDP**

Policy name	Type	Attached via
AmazonEC2FullAccess	AWS managed	Directly
AmazonRoute53FullAccess	AWS managed	Directly
AmazonRoute53ResolverFullAccess	AWS managed	Directly
AmazonSSMFullAccess	AWS managed	Directly
AmazonSSMGUIConnect	Customer inline	Inline
AmazonSSMManagedInstanceCore	AWS managed	Directly
AmazonVPCFullAccess	AWS managed	Directly
EC2InstanceConnect	AWS managed	Directly
IAMReadOnlyAccess	AWS managed	Directly

Reminder: AWS recommends reserving the root account for billing and account-level changes only.

Step 1.4 Enable Billing Alerts (Recommended)

18. In the console search bar, type Billing and open Billing and Cost Management.
19. Go to Billing Preferences and enable Receive Free Tier Usage Alerts, entering your email.
20. Optionally, go to Budgets and create a zero-spend budget for extra peace of mind.

Task 2 — Launch an EC2 Instance, Configure VPC Networking and Security Groups, and Connect Remotely

This task builds a custom VPC and public subnet, then splits into two tracks: Task 2.1 launches an Ubuntu instance reachable over SSH, and Task 2.2 launches a Windows instance reachable over RDP. Complete 2.1 first — its VPC and subnet are reused in 2.2.

Step 2.1 Resource Reference Table

Use this table as a quick reference throughout Task 2:

Resource	Name / Value	Notes
VPC	demo-vpc	CIDR: 10.0.0.0/16
Public Subnet	demo-public-subnet	CIDR: 10.0.1.0/24
Internet Gateway	demo-igw	Attached to VPC
Route Table	demo-public-rt	0.0.0.0/0 → IGW
Security Group (SSH)	ssh-web-sg	SSH + HTTP inbound
Security Group (RDP)	rdp-sg	RDP inbound
Key Pair (Linux)	ubuntu-key	.pem file — keep this safe!
Key Pair (Windows)	windows-key	.pem file used to decrypt the password

Task 2.1 — SSH Access Using an Ubuntu Instance

Step 2.1.1 Create a Custom VPC

21. Sign in to the AWS Console and select your preferred Region from the top-right dropdown (e.g., US East (N. Virginia) us-east-1).
22. Search for VPC in the console search bar and open the VPC service.
23. Click Create VPC. Choose VPC only, configure:

Field	Value to Enter
Resources to create	VPC only
Name tag	demo-vpc
IPv4 CIDR block	10.0.0.0/16
IPv6 CIDR block	No IPv6 CIDR block
Tenancy	Default

24. Click Create VPC, then Close.
25. In the left panel, click Subnets → Create subnet. Configure:

Field	Value to Enter
VPC ID	Select demo-vpc
Subnet name	demo-public-subnet
Availability Zone	No preference
IPv4 subnet CIDR block	10.0.1.0/24

26. Click Create subnet.
27. Select demo-public-subnet, click Actions → Edit subnet settings, enable Auto-assign public IPv4 address, and save.
28. Click Internet Gateways → Create internet gateway, name it **demo-igw**, then create. Select it, click Actions → Attach to VPC, and attach it to **demo-vpc**.
29. Click Route Tables → Create a route table, name it **demo-public-rt**, associate it with **demo-vpc**, and create.
30. Open demo-public-rt, go to the Routes tab, click Edit routes → Add route: destination 0.0.0.0/0, target the **demo-igw internet gateway**, then save.
31. Open the Subnet associations tab, click Edit subnet associations, select demo-public-subnet, and Save.

☑ **Routed!** Your public subnet now has a path to the internet through the internet gateway.

Step 2.1.2 Create a Security Group for SSH

32. In the VPC or EC2 console, click Security Groups → Create security group.
33. Name it ssh-web-sg, add a description, and select demo-vpc as the VPC.
34. Add inbound rules:

Type	Protocol	Port Range	Source / Purpose
SSH	TCP	22	My IP (recommended) — remote shell access
HTTP	TCP	80	0.0.0.0/0 — required for Task 3 web access

35. Leave Outbound rules at their default and click Create security group.

⚠ **Careful: opening SSH (port 22) to 0.0.0.0/0 exposes the instance to brute-force attempts from the whole internet. Restrict to My IP whenever possible.**

Step 2.1.3 Create a Key Pair

36. Open the EC2 console and click Key Pairs → Create key pair.
37. Configure:

Field	Value
Name	ubuntu-key
Key pair type	RSA
Private key file format	.pem (Mac/Linux/OpenSSH) or .ppk (PuTTY)

38. Click Create key pair — the private key file downloads automatically. AWS does not keep a copy, so store it securely.

```
# On macOS/Linux, restrict permissions so SSH will accept the key:
chmod 400 ubuntu-key.pem
```

Step 2.1.4 Launch the Ubuntu EC2 Instance

39. In the EC2 console, click Launch Instance.
40. Enter a Name, e.g., ubuntu-web-server.
41. Under Application and OS Images (AMI), select Ubuntu Server 24.04 LTS (Free tier eligible).
42. Under Instance type, select t2. micro or t3. micro (Free tier eligible).
43. Under Key pair (login), select ubuntu-key.

44. Under Network settings, click Edit. Select demo-vpc, demo-public-subnet, and set Auto-assign public IP to Enable.
45. Under Firewall (security groups), select the existing security group ssh-web-sg.
46. Leave Configure storage at the default 8 GiB gp3 volume.
47. Review the summary and click Launch instance.
48. Click View all instances and wait until Instance State shows Running and Status check shows 2/2 checks passed (1–2 minutes).
49. Select the instance and note its Public IPv4 address — you'll need it to connect.

Step 2.1.5 Connect via SSH

Choose the option matching your operating system.

Option A — macOS / Linux / Windows 10+ (built-in OpenSSH client)

```
cd /path/to/downloaded/key
chmod 400 ubuntu-key.pem [Applicable for mac/Linux]
#For windows OS perform the following operations:
icacls .\ubuntu-key.pem /inheritance:ro
icacls .\ubuntu-key.pem /grant:r "$($env:USERNAME):(R)"
ssh -i ubuntu-key.pem ubuntu@<PUBLIC_IP_OR_DNS>
```

Replace <PUBLIC_IP_OR_DNS> with the instance's public IPv4 address or public DNS name. Type yes when prompted to accept the host's fingerprint.

Option B — Windows using PuTTY

- Download and install PuTTY and PuTTYgen from the official putty.org site.
- Open PuTTYgen, click Load, select the .pem file (set filter to All Files), then click Save private key to produce a .ppk file.
- Open PuTTY, enter the public IP under Host Name; under Connection → SSH → Auth → Credentials, browse to the .ppk file.
- Click Open, and when prompted for a username, enter ubuntu.

> 📌 **Tip:** the default login username depends on the AMI: "ubuntu" for Ubuntu, "ec2-user" for Amazon Linux, and "admin" for Debian.

On success, you'll see the Ubuntu welcome banner and a shell prompt like:

```
ubuntu@ip-10-0-1-XX:~$
```

Task 2.2 — RDP Access Using a Windows Instance

This track reuses the demo-vpc and demo-public-subnet from Task 2.1. Only the security group, key pair, AMI, and connection method differ.

Step 2.2.1 Create a Security Group for RDP

50. In the EC2 or VPC console, click Security Groups → Create security group.
51. Name it rdp-sg, add a description, and select **demo-vpc** as the VPC.
52. Add an inbound rule:

Type	Protocol	Port Range	Source / Purpose
RDP	TCP	3389	My IP (recommended) — remote desktop access

53. Leave Outbound rules at default and click Create security group.

Step 2.2.2 Create a Key Pair for Windows

54. In the EC2 console, click Key Pairs → Create key pair.
55. Name it **windows-key**, set type to RSA and format to **.pem**, then create a key pair. These key decrypts the administrator password; it is not used for interactive login.

Step 2.2.3 Launch the Windows EC2 Instance

56. In the EC2 console, click Launch Instance and name it windows-web-server.
57. Under Application and OS Images (AMI), search for and select **Microsoft Windows Server 2025 Base** (Free tier eligible).
58. Under Instance type, select t2.micro or t3.micro (Free tier eligible).
59. Under Key pair (login), select Windows Key.
60. Under Network settings, click Edit, choose demo-vpc and demo-public-subnet, set Auto-assign public IP to Enable, and select the existing rdp-sg security group.
61. Leave storage at default settings and click Launch instance.
62. Wait until the instance shows Running and 2/2 checks passed (Windows can take 3–5 minutes).

Step 2.2.4 Retrieve the Administrator Password

63. Select the running Windows instance, click Connect, and open the RDP client tab.

64. Click Get password.
65. Click Upload private key file, select windows-key.pem, and click Decrypt Password.
66. Copy and securely store the decrypted Administrator password.

Step 2.2.5 Connect via RDP

Option A — Windows (built-in Remote Desktop Connection)

- Press Win + R, type mstsc, and press Enter.
- Enter the instance's public IP and click Connect.
- When prompted, enter username Administrator and paste the decrypted password.
- Accept the self-signed certificate warning to reach the desktop.

If it does not work, then we have to perform the following steps:

Step 1: Sign out of your current AWS IAM account

- Click on your account name in the top-right corner.
- Click **Sign out**.

Step 2: **Log in as the Root User**

- Go to the AWS sign-in page.
- Select **Root user**.
- Enter the email address associated with your AWS account.
- Click **Next**.
- Enter the root user password.
- Complete MFA (if enabled).

Step 3: Create an IAM role

- In the AWS Console search bar, type **IAM**.
- Open the **IAM** service.
- In the left menu, click **Roles**.
- Click **Create role**.

Step 4: Select the Trusted Entity

On the **"Select trusted entity"** page:

- **Trusted entity type:** AWS service
- **Service or use case:** EC2

Click **Next**.

Step 5: Attach the Required Policy

In the search box, type:

AmazonSSMManagedInstanceCore

Check the box next to:

AmazonSSMManagedInstanceCore

Click Next.

Step 6: Name the Role

- Enter: EC2-SSM-Role
- Click **Create role**.
- You should now see the role in the list.

Step 7: Attach the Role to Your EC2 Instance

- Go to the **EC2** console.
- Click Instances.
- Select your Windows instance.
- Click: Actions → Security → Modify IAM role

Step 8: Select the IAM Role

- From the drop-down list, choose: EC2-SSM-Role
- Click: Update IAM role
- You should see a success message.

Step 9: Wait for the SSM Agent

- Wait about **3–5 minutes**.

Step 10: Check System Manager

In the AWS Console:

- Search for **Systems Manager**.
- Open **Systems Manager**.
- Go to: **Fleet Manager** (Available on the left side panel)
- Your EC2 instance should appear as **Managed**.

Step 11: If it doesn't appear

- Reboot the EC2 instance: EC2 → Instances → Select your instance → Instance State → Reboot Instance
- Wait another **5 minutes** and check **Managed Nodes** again.

Step 12: If it appears

- Click on the Node ID
- On the top right, there is a drop-down button **"Node Actions"**, move the pointer there and select:
    - Node Actions → Connect → Connect with Remote Desktop
- Provide the username and password received after decrypting the .pem key of Windows, and click **"connect"**

Step 13: Now, also add the customized policy to the IAM account so that, from IAM account we can conncet using the fleet manager to the GUI interface

- **Grant Only the Required Permissions (Recommended)**
    - If you prefer least-privilege access, create a custom IAM policy with the following JSON:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "FleetManagerGUIConnect",
      "Effect": "Allow",
      "Action": [
        "ssm-guiconnect:StartConnection",
        "ssm-guiconnect:GetConnection"
      ],
      "Resource": [
        "arn:aws:ec2:*:*:instance/*"
      ]
    },
    {
      "Sid": "SessionManager",
      "Effect": "Allow",
      "Action": [
        "ssm:StartSession",
        "ssm:ResumeSession",
        "ssm:TerminateSession",
        "ssm:DescribeSessions",
        "ssm:GetConnectionStatus",
        "ssm:DescribeInstanceInformation"
      ],
      "Resource": "*"
    },
    {
      "Sid": "SSMMessages",
      "Effect": "Allow",
      "Action": [
        "ssmmessages:CreateControlChannel",
        "ssmmessages:CreateDataChannel",
        "ssmmessages:OpenControlChannel",
        "ssmmessages:OpenDataChannel"
      ],
      "Resource": "*"
    },
    {
      "Sid": "EC2Describe",
      "Effect": "Allow",
      "Action": [
        "ec2:DescribeInstances",
        "ec2:DescribeInstanceStatus"
      ],
      "Resource": "*"
    }
  ]
}
```

- The path to write the above policy is: Login to Root Account → Search IAM Users → Select the IAM Account → In permission policies →Select the "Create the Inline Policy" →Select JSON →Paste the above Script→Click Next →Give the name to the policy "AmazonSSMGUIConnect"→save

- Now, log out from the root account and log in to the IAM account, select the EC2 instance, and click Connect. It will open the CLI-based systems

- For GUI→Search Fleet Manager → Select the Node ID, →click on "Node Actions" → Connect → Connect with Remote Desktop, provide username and password (or) use the option "Key pair" →Upload the .pem key, and click-on "Connect " button.

Option B —a macOS

- Install Microsoft Remote Desktop from the Mac App Store.
- Add a new PC using the public IP as the PC name.
- Add a user account (Administrator + decrypted password), then double-click to connect.

Option C — Linux

- Install an RDP client, e.g., Remmina (sudo apt install remmina) or FreeRDP.
- Create a connection profile with the public IP, username Administrator, and the decrypted password.

⚠ If it times out: confirm the security group's RDP rule source matches your current public IP, and that status checks have fully passed.

Task 3 — Install and Configure a Web Server, deploy a Web Application, and Verify Accessibility

This task can be performed on either the Ubuntu instance (Apache or Nginx, accessed via the SSH session from Task 2.1) or the Windows instance (IIS, accessed via the RDP session from Task 2.2).

Step 3.1 Update the System (Ubuntu)

67. From your SSH session, update the package index and upgrade packages:

```
sudo apt update && sudo apt upgrade -y
```

Step 3.2 Option A — Install and Configure Apache

68. Install Apache:

```
sudo apt install apache2 -y
```

69. Start and enable the service:

```
sudo systemctl start apache2
sudo systemctl enable apache2
```

70. Verify it's active:

```
sudo systemctl status apache2
```

71. If ufw is enabled, allow HTTP traffic (in addition to the AWS security group rule):

```
sudo ufw allow "Apache"
sudo ufw status
```

Step 3.3 Option B — Install and Configure Nginx

72. Install Nginx:

```
sudo apt install nginx -y
```

73. Start and enable the service:

```
sudo systemctl start nginx
sudo systemctl enable nginx
```

74. Verify it's active:

```
sudo systemctl status nginx
```

75. If ufw is enabled, allow HTTP traffic:

```
sudo ufw allow "Nginx HTTP"
sudo ufw status
```

📌 **Note:** install only one of Apache or Nginx — both bind to port 80 by default and will conflict if run together.

Step 3.4 Deploy a Simple Web Application

76. Navigate to the default web root (Apache and Nginx both default to /var/www/html):

```
cd /var/www/html
```

77. Back up and remove the default placeholder page:

```
sudo mv index.html index.html.bak 2>/dev/null
```

78. Create a new index.html:

```
sudo nano index.html
```

79. Paste the following, then save and exit (Ctrl+O, Enter, Ctrl+X):

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>My AWS Web App</title>
</head>
<body style="font-family: Arial, sans-serif; text-align:center; margin-top:80px;">
  <h1>Hello from my AWS EC2 Instance!</h1>
```

```
  <p>This page confirms Apache/Nginx is running and reachable.</p>
</body>
</html>
```

80. Set correct ownership and permissions:

```
sudo chown -R www-data:www-data /var/www/html
sudo chmod -R 755 /var/www/html
```

81. Restart the web server to serve the new content cleanly:

```
sudo systemctl restart apache2   # or: sudo systemctl restart nginx
```

Step 3.5 Windows Path — IIS (Optional Alternative)

82. From the RDP session, open Server Manager → Manage → Add Roles and Features.
83. Choose Role-based installation → local server → check Web Server (IIS) → accept dependencies → Install.
84. Open File Explorer, navigate to C:\inetpub\wwwroot.
85. Rename iisstart.htm, then create index.html with the same HTML from Step 3.4 using Notepad.
86. Open IIS Manager, expand Sites → Default Web Site, and confirm index.html is listed under Default Document (move it to the top if needed).
87. In the Actions pane, click Restart on the Default Web Site.

📌 **Networking reminder:** IIS needs the same HTTP (TCP 80) inbound rule already added to ssh-web-sg — add an equivalent to rdp-sg if deploying on Windows.

Step 3.6 Verify Accessibility via Public IP

88. In the EC2 console, select your running instance and copy its Public IPv4 address.
89. Open a browser and navigate to:

```
http://<PUBLIC_IP>
```

90. Confirm the custom "Hello from my AWS EC2 Instance!" page (or IIS equivalent) loads successfully.
91. Optionally, verify from the command line:

```
curl -I http://<PUBLIC_IP>
```

Success: an "HTTP/1.1 200 OK" response confirms the web server is reachable and serving content.

Tip: A non-Elastic public IP changes if the instance is stopped and restarted. Allocate an Elastic IP if you need a persistent address.

Step 3.7 Troubleshooting

Problem	Likely Cause	Solution
The page doesn't load in the browser	Security group missing HTTP rule	Add an inbound rule for TCP port 80 from 0.0.0.0/0 (Step 2.1.2)
Connection times out	No route to the internet	Verify the route table has 0.0.0.0/0 → the internet gateway (Step 2.1.1)
curl/browser: connection refused	Web service not running	Run sudo systemctl status apache2 (or nginx) and restart if needed
SSH permission denied (publickey)	Wrong key or bad permissions	Run chmod 400 on the .pem file; confirm the correct key pair
RDP won't connect	Security group source IP mismatch	Confirm the RDP rule's source matches your current public IP

Step 3.8 Cleanup (Recommended After Testing)

- Terminate EC2 instances: EC2 console → Instances → Instance State → Terminate.
- Release any Elastic IPs: VPC console → Elastic IPs → Release.
- Delete the custom VPC, subnets, route tables, and internet gateway if no longer needed.
- Delete unused key pairs and security groups.

Reminder: Failing to delete resources can result in ongoing charges even if instances are stopped.